

Banco de Dados no Mercado Corporativo de TI



O Modelo Teórico vs. A Realidade em Produção

Abordagem Acadêmica

Normalização 3NF: Eliminação absoluta de redundância.

Servidor Central: Premissa de nó transacional isolado.

Integridade Estrita: Travamento via constraints e triggers.

Prática Corporativa

Desnormalização: Viabilidade de leitura em milissegundos.

Clusters Distribuídos: Réplicas e partição geográfica.

Custo em Cloud: Otimização agressiva de IOPS e RAM.

O Modelo Teórico vs. A Realidade em Produção

FuncN	Nome	Cargo	Salário	NumAg	Endereço
25	Luiz	Caixa	2000	1632	Prudente de Moraes, 15
30	Ricardo	Gerente	5000	1668	Hermes da Fonseca, 20
31	Josita	Caixa	2000	1632	Prudente de Moraes, 15
32	Francisca	Gerente	5600	1632	Prudente de Moraes, 15
33	Andréia	Caixa	2300	1668	Hermes da Fonseca, 20

Através da Chave primária é possível obter os valores dos outros atributos

No entanto, eu também posso obter o endereço da Agência pelo atributo NumAg. Isso viola a 3FN e por esse motivo a tabela não está na 3FN.



Query

```
SELECT
    o.id AS order_id,
    o.created_at,
    o.status,
    c.full_name AS customer_name,
    c.email,
    c.tax_id AS customer_cpf,
    a.street,
    a.number,
    ci.name AS city_name,
    st.uf AS state_uf,
    SUM(oi.quantity * p.price) AS total_amount,
    COUNT(oi.id) AS total_items
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
INNER JOIN customer_addresses ca ON ca.customer_id = c.id AND ca.is_default = true
INNER JOIN addresses a ON ca.address_id = a.id
INNER JOIN cities ci ON a.city_id = ci.id
INNER JOIN states st ON ci.state_id = st.id
INNER JOIN order_items oi ON oi.order_id = o.id
INNER JOIN products p ON oi.product_id = p.id
WHERE o.tenant_id = 42
GROUP BY
    o.id, o.created_at, o.status,
    c.full_name, c.email, c.tax_id,
    a.street, a.number, ci.name, st.uf
ORDER BY o.created_at DESC
LIMIT 20;
```

Persistência Poliglota na Indústria

Durante décadas, a arquitetura de software tradicional sofreu do antipadrão "**Golden Hammer**" (**Martelo de Ouro**): a crença de que um único banco de dados relacional genérico deveria resolver todas as necessidades de um sistema:

- Dados transacionais de pagamento? SQL.
- Sessão de usuário? SQL.
- Logs e auditoria? SQL.
- Busca textual? SQL (``LIKE %termo%``).

Persistência Poliglota na Indústria



Relacional (PostgreSQL)

Transações financeiras auditáveis.



Chave-Valor (Redis)

Cache ultrarrápido em memória.



Busca Textual (Elastic)

Consultas textuais complexas.

Persistência Poliglota na Indústria

ACID vs BASE

Característica	ACID	BASE
Significado	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventual consistency
Consistência	Forte	Geralmente eventual
Disponibilidade	Pode ser sacrificada em algumas situações	Prioriza disponibilidade
Transações	Fortes e bem definidas	Mais flexíveis/distribuídas
Dados	Precisam estar consistentes após a transação	Podem ficar temporariamente inconsistentes
Escalabilidade horizontal	Pode ser mais complexa	Geralmente favorecida
Exemplo típico	Bancos relacionais	Sistemas distribuídos/NoSQL

Persistência Poliglota na Indústria

No VoleiPlay, cada motor tem um propósito cirúrgico:

Motor	Tipo	Tecnologia	Papel no VoleiPlay	Garantia Principal
 PostgreSQL 16	Relacional Leve	SQL / Tabelas	Gestão contábil do torneio, premiações da Saga e atletas estruturados	ACID Estrito
 MongoDB 7	NoSQL Documental	BSON / JSON	Scouts de partidas, lances, mapa de calor na areia e dados aninhados	Flexibilidade (BASE)
 Redis 7	In-Memory Key-Value	Memória RAM	Caching de alta velocidade (Cache-Aside), sessões e invalidação rápida	Latência Sub-Milissegundo
 Elasticsearch 7	Search Engine	Inverted Index	Busca fonética de atletas e agregação de telemetria APM	Full-Text Search & Analytics

SQL Corporativo: Confiabilidade e Rigor

Casos de Uso Corporativos

Livro-Razão Financeiro

Transações monetárias com auditoria matemática absoluta.

Sistemas ERP & CRM

Relacionamentos complexos com integridade referencial nativa.

Especificidades Técnicas

Transações ACID: Garantia integral de atomicidade e consistência estrita.

Concorrência MVCC: Leituras não bloqueiam transações de escrita concorrentes.

Escala Vertical: Expansão primária através de CPU e memória.

Estruturas B-Tree: Índices balanceados para busca e ordenação rápida.

Redis: Baixa Latência e Concorrência

Casos de Uso Corporativos

Gestão de Sessões & Tokens

Validação de autenticação com expiração automática TTL.

Rate Limiting de APIs

Controle estrito de vazão para conter ataques de negação.

Especificidades Técnicas

Single-Threaded: Event loop elimina bloqueios e locks concorrentes.

RAM Nativa: Respostas ultrarrápidas em microssegundos para consultas.

Tipos Avançados: Manipulação atômica de Hashes, Sets e Streams.

Persistência Híbrida: Gravação em disco via snapshots RDB e AOF.

Redis: Baixa Latência e Concorrência

3 Armadilhas Clássicas de Caching

1. **Cache Stampede (Efeito Boiada / Thundering Herd):**

- *Cenário:* O atleta campeão olímpico tem milhões de acessos por segundo. O TTL da sua chave de cache expira.
- *Problema:* Instantaneamente, 50.000 requisições simultâneas recebem MISS e disparam a mesma query pesada no PostgreSQL ao mesmo tempo, derrubando o banco relacional.
- *Solução:* Mutex/Lock distribuído com Redis para que apenas 1 requisição vá ao banco enquanto as outras aguardam a renovação do cache.

Redis: Baixa Latência e Concorrência

2. Cache Avalanche (Queda Simultânea):

- *Cenário*: O desenvolvedor colocou $TTL = 3600$ (exatamente 1 hora) para todos os 100.000 jogadores na hora do boot.
- *Problema*: Após exatamente 60 minutos, todos os 100.000 itens expiram no mesmo segundo. O banco recebe uma avalanche de queries.
- *Solução*: **Jitter aleatório** no TTL (ex: $TTL = 3600 + \text{random}(-300, 300)$).

Redis: Baixa Latência e Concorrência

3. Cache Penetration (Penetração de Cache):

- *Cenário*: Um invasor envia milhões de requisições buscando IDs inexistentes (/players/id-fantasma-999999).
- *Problema*: Como o ID não existe, o Redis nunca grava o dado. Toda requisição atinge diretamente o banco de dados.
- *Solução*: Gravar chaves vazias com TTL curto no Redis (SET key null EX 30).

NoSQL: Schema Flexível e Escala Horizontal



Casos de Uso Corporativos

Catálogo de E-Commerce

Itens com variações heterogêneas sem travar schema relacional.

Payloads de Integração

Ingestão contínua de eventos brutos de APIs externas.



Especificidades Técnicas

Escala Horizontal: Distribuição automática de dados através de sharding.

Schema Dinâmico: Documentos BSON flexíveis sem migrations de tabelas.

Consistência Ajustável: Ajuste direto entre latência e réplicas confirmadas.

Desnormalização Nativa: Dados aninhados eliminam a sobrecarga de joins.

Elasticsearch: Busca e Análise em Escala

Casos de Uso Corporativos

Mecanismo de Busca

Autocomplete corporativo com tolerância a erros e typos.

Observabilidade de Logs

Ingestão e análise em tempo real na stack ELK corporativa.

Especificidades Técnicas

Índice Invertido: Mapeamento de termos para recuperação instantânea.

Scoring com BM25: Classificação por relevância contextual e frequência estatística.

Cluster Distribuído: Divisão em nós mestres e nós de dados.

Agregações Massivas: Cálculos analíticos instantâneos sobre milhões de registros.

O SQL Moderno Superou a Dicotomia?

JSONB NATIVO

Schemas Semiestruturados

Flexibilidade NoSQL com ACID.

PGVECTOR & IA

Busca Semântica Integrada

Embeddings dentro do banco SQL.

PARTICIONAMENTO

Tabelas em Escala Terabyte

Divisão transparente por tempo.

Problemas que eu experienciei e a solução

1. A tabela não suportava mais dados

- Analise
- Impacto
- Cópia
- Expurgo

Problemas que eu experienciei e a solução

2. Muitas requisições derrubavam um banco terceiro

- Analise
- Impacto
- Cache falso com NoSQL
- TTL

Problemas que eu experienciei e a solução

3. Triggers sem conhecimento para auditoria

- Analise
- Impacto
- Atualizar script de trigger

Problemas que eu experienciei e a solução

4. Valor de cache em MB

- Analise
- Impacto
- Alterar o ofensor
- Apagar dado

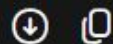
Problemas que eu experienciei e a solução

5. Migration travando banco de dados

- Analise
- Impacto
- Evitar proteções como FK
- Criar regras para isso

A Armadilha do Problema N+1

TypeScript



```
// 1. Busca os 100 usuários
const users = await userRepository.find({ take: 100 });

// 2. Itera sobre cada usuário para pegar o perfil
const resultado = users.map(user => {
  return {
    nome: user.name,
    perfil: user.profile.roleName // <-- O ORM dispara uma query oculta aqui!
  };
});
```

Impacto de 100 Pedidos

Gera 101 conexões e requisições.

Consulta Otimizada

Apenas 1 round-trip no banco.

A Armadilha do Problema N+1

Por que isso é uma "Armadilha" tão grave?

1. **Invisibilidade no ambiente local:**

- Na máquina do desenvolvedor (onde o banco roda localmente em **localhost** com latência de ping de 0,1 ms e apenas 5 usuários de teste), 6 queries rodam em 2 milissegundos. Ninguém percebe o problema.
- Em produção, o servidor da aplicação está em uma máquina e o banco em outra (ex.: AWS RDS). Se a latência de rede entre eles for de 3 ms (ida e volta), só de tempo de tráfego de rede (handshakes e round-trips) temos:

100 requests x 3ms = 300ms a mais!

2. **Esgotamento do Connection Pool:**

- Se 20 usuários acessarem essa página ao mesmo tempo, são $20 \times 101 = 2020$ queries disparadas em fração de segundo.
- As conexões do banco se esgotam (**too many connections**), a fila de espera trava e a aplicação inteira cai.

O Esgotamento de Conexões em Nuvem

100

Limite padrão de conexões simultâneas em instâncias RDS intermediárias

Gargalo Serverless

Cada função lambda cria conexões.

Solução com PgBouncer

Pool reutiliza conexões abertas.

Diagnóstico Prático de Gargalos

01

Métricas de Servidor

Saturação de CPU ou disco.

02

Explain Analyze

Detecção de Sequential Scans.

03

Indexação Estratégica

Criação de índices compostos.

04

Auditoria de Locks

Identificação de transações longas.

Disaster Recovery: Métricas Essenciais



RPO (Recovery Point)

Tolerância máxima de perda temporal.

RTO (Recovery Time)

Janela temporal até restaurar serviço.

Multi-AZ Automático

Failover em segundos entre zonas.

Réplicas e o Dilema do Replication Lag

Replicação Síncrona

Garantia RPO Zero: Dados confirmados em todos nós.

Custo em Latência: Escrita aguarda confirmação remota.

Replicação Assíncrona

Escrita Imediata: Resposta sem bloquear aplicação.

Risco de Descompasso: Leituras obsoletas em réplicas.

Backup Não é Disaster Recovery

Snapshots Intestados

Backups sem teste de restore falham.

Point-in-Time Recovery

Restauração contínua minuto a minuto.

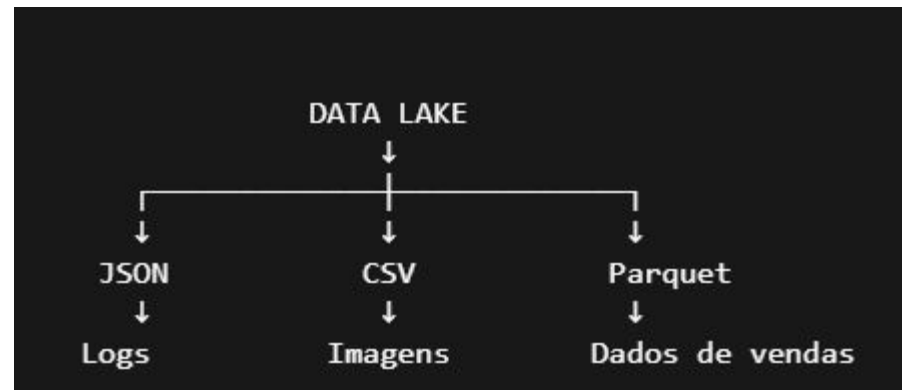
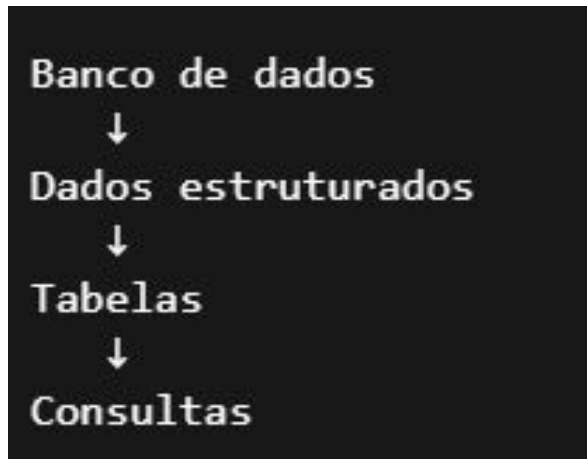
Corrupção Lógica

Migrations erradas propagam nas réplicas.

Usos mistos

Data Lake

Repositório de dados para Big Data, geralmente baseado em armazenamento de objetos ou sistemas de arquivos distribuídos.



Ideias Práticas para a Sala de Aula

Planos de Execução Obrigatórios: Avaliar consultas via EXPLAIN ANALYZE nas tarefas.

Simulação de Carga com k6: Exibir o esgotamento real de pool de conexões.

Modelagem Acesso-Primeiro: Desafiar os alunos a desenharem esquemas pensando em IOPS.

Laboratórios de Resiliência: Simulação manual de failover entre nós primário e réplica.

CONCLUSÃO & DISCUSSÃO

Duvidas?

